



N10 Verteilte Systeme

IoT Sensor Dashboard

Teilnehmer:	Ayoub Guedria
Semester:	SoSe 2026
Dozent:	Prof. Dr.-Ing. Alexandra Mikityuk
Abgabedatum:	14.07.2026

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielsetzung	1
1.3	Projektüberblick	1
2	Grundlagen	2
2.1	ESP32 und MicroPython	2
2.2	MQTT-Protokoll	2
2.3	REST API	3
2.4	Sensoren	3
2.4.1	BME280	3
2.4.2	MQ-2	3
3	Implementierung	4
3.1	Blockschaltbild	4
3.2	Hardware- und MicroPython-Setup	5
3.2.1	ESP32	5
3.2.2	BME280 Verkabelung	6
3.2.3	MQ-2 Verkabelung	6
3.2.4	MicroPython flashen	7
3.3	Docker Deployment	8
3.4	Cloud Deployment	9
3.4.1	Railway Deployment	9
3.4.2	HiveMQ Cloud und MQTT Authentication	10
4	Ergebnisse	11
4.1	Echte Sensordaten	11
4.1.1	BME280-Sensoren	11
4.1.2	MQ-2-Sensor	11
4.1.3	Serial Monitor	12
4.2	Mehrere Sensoren	13
4.3	Benutzer-Login	13
4.4	Echtzeit Auto Refresh	14
4.5	MQTT Authentication	15

4.6	Historische Analyse	15
4.7	Grafische Darstellung mit Diagrammen	16
4.8	Messwert-Tabellen	17
4.8.1	BME280-Messwert-Tabelle	17
4.8.2	MQ-2-Messwert-Tabelle	17
4.9	Grenzwertwarnungen	18
4.10	Machine-Learning-Vorhersage	18
4.11	E-Mail-Alerts	19
4.11.1	Temperaturwarnung	20
4.11.2	Gas- und Rauchwarnung	20
4.11.3	Machine-Learning-Warnung	22
4.12	CSV/PDF-Export	22
4.12.1	CSV-Export	22
4.12.2	PDF-Export	22
5	Herausforderungen	25
5.1	ESP32 Flash- und COM-Port-Probleme	25
5.2	Lokaler MQTT-Broker nicht erreichbar	25
5.3	MQTT Authentication	25
5.4	Falsches MQTT Topic	25
5.5	Fehlender Timestamp	26
5.6	Backend empfing Daten, aber /sensordaten war leer	26
5.7	Dashboard leer trotz Backend-Daten	26
5.8	Zeitdarstellung	26
5.9	BME280 Temperaturabweichung	26
5.10	MQ-2 Test und Sicherheit	27
5.11	E-Mail-Alerts	27
5.12	CSV/PDF Export	28
6	Fazit und Ausblick	28
	Abbildungsverzeichnis	30
	Literatur	31
A	ESP32 Befehle	32

A.1	MicroPython flashen	32
A.2	Dateien auf ESP32 kopieren	32
B	Docker und Backend Tests	32
B.1	Docker lokal starten	32
B.2	Backend testen	32
B.3	Streamlit testen	32
C	Wichtige Konfigurationen	32
C.1	Railway Backend Variables	32
C.2	Railway Frontend Variables	33
D	Python-Hauptprogramm: IoT Sensor Dashboard	33
D.1	Wichtige Ausschnitte aus dem Backend-Hauptprogramm	33

1 Einleitung

1.1 Motivation

In modernen Anwendungen des Internet of Things werden Messwerte nicht mehr nur lokal erfasst, sondern über Netzwerke zwischen mehreren unabhängigen Komponenten übertragen. Dadurch entstehen verteilte Systeme, bei denen Sensoren, Broker, Backend, Datenverarbeitung und Frontend getrennt voneinander laufen und trotzdem zusammenarbeiten müssen.

Die Motivation dieses Projekts war der Aufbau eines realistischen IoT-Systems mit echter Hardware. Statt nur simulierte Daten zu verwenden, sollten echte Sensordaten von einem ESP32 DevKit erfasst und über MQTT an ein Online-Dashboard übertragen werden. Dadurch konnte praktisch gezeigt werden, wie verteilte Komponenten über Netzwerkprotokolle miteinander kommunizieren.

1.2 Zielsetzung

Ziel des Projekts war die Entwicklung eines vollständigen IoT Sensor-Dashboards mit folgenden Funktionen:

- Erfassung echter ESP32-Sensordaten
- Unterstützung mehrerer Sensoren
- MQTT-Kommunikation über Publish/Subscribe
- MQTT Authentication zur Absicherung der Datenübertragung
- REST API zur Bereitstellung der Sensordaten
- Online Dashboard mit Auto Refresh
- Diagramme und historische Analyse
- Grenzwertwarnungen für kritische Messwerte
- E-Mail-Alerts bei Warnungen
- CSV/PDF Export
- Docker Deployment
- Cloud Deployment
- Machine-Learning-Vorhersage

1.3 Projektüberblick

Das Projekt besteht aus mehreren verteilten Komponenten. Der ESP32 liest echte Sensorwerte von zwei BME280-Sensoren und einem MQ-2-Sensor aus. Die Daten werden im JSON-Format an einen MQTT-Broker gesendet. Das Backend empfängt die MQTT-Nachrichten, verarbeitet und speichert sie. Anschließend stellt es die Daten über eine

REST API bereit. Das Frontend ruft diese API auf und visualisiert die Daten im Browser.

Die finale Datenkette lautet:

```
1 ESP32 Sensoren -> HiveMQ Cloud MQTT Broker -> Railway Backend -> Railway Frontend Dashboard
```

2 Grundlagen

2.1 ESP32 und MicroPython

Der ESP32 DevKit ist ein Mikrocontroller mit WLAN-Unterstützung. Er eignet sich gut für IoT-Projekte, weil er Sensoren auslesen und Daten direkt über Netzwerkprotokolle senden kann [5].

Der ESP32 wurde in diesem Projekt mit MicroPython programmiert. MicroPython ist eine spezielle und kleinere Python-Version für Mikrocontroller. Dadurch kann Python-ähnlicher Code direkt auf dem ESP32 ausgeführt werden. Das ist besonders wichtig, weil der ESP32 weniger Speicher und Rechenleistung als ein normaler Computer oder Server besitzt.

Im Projekt wurden Python und MicroPython verwendet. Das Backend und das Frontend wurden mit normalem Python entwickelt. Der ESP32 wurde dagegen mit MicroPython programmiert. MicroPython ermöglicht den direkten Zugriff auf Hardwarefunktionen wie GPIO, I2C, ADC und WLAN. Dadurch konnte der ESP32 die BME280- und MQ-2-Sensoren auslesen und die Messwerte per MQTT an den Broker senden.

Zusammengefasst läuft in diesem Projekt MicroPython auf dem ESP32, während normales Python für Backend und Frontend verwendet wird:

```
1 ESP32                -> MicroPython
2 Backend / FastAPI    -> Python
3 Frontend / Streamlit -> Python
```

2.2 MQTT-Protokoll

MQTT ist ein leichtgewichtiges Publish/Subscribe-Protokoll. Es eignet sich besonders für IoT-Systeme, weil Sensorgeräte Daten an Topics veröffentlichen können, ohne direkt zu wissen, wer diese Daten empfängt.

In diesem Projekt veröffentlicht der ESP32 Sensordaten an folgende Topics:

```
1 sensors/bme280_1
2 sensors/bme280_2
3 sensors/mq2_1
```

Das Backend abonniert alle Sensor-Topics mit:

```
1 sensors/#
```

Das Zeichen # ist ein MQTT-Wildcard-Zeichen. `sensors/#` bedeutet, dass alle Untertopics von `sensors/` empfangen werden, zum Beispiel `sensors/bme280_1`, `sensors/bme280_2` und `sensors/mq2_1`.

Für den Cloud-Betrieb wurde HiveMQ Cloud verwendet [1].

2.3 REST API

Eine REST API stellt Daten über HTTP-Endpunkte bereit. In diesem Projekt verwendet das Dashboard die REST API, um aktuelle und historische Sensordaten vom Backend abzurufen.

Wichtige Endpunkte:

```
1 GET /health
2 GET /sensordaten
3 GET /history
4 GET /sensor-data
5 POST /ml-alert-email
```

Der wichtigste Endpoint ist:

```
1 https://backend-production-4878.up.railway.app/sensordaten
```

Das Backend wurde mit FastAPI umgesetzt [2]. Das Dashboard wurde mit Streamlit entwickelt [3].

2.4 Sensoren

2.4.1 BME280

Der BME280 misst Temperatur, Luftfeuchtigkeit und Luftdruck [7]. Im Projekt wurden zwei BME280-Sensoren verwendet. Die erkannten I2C-Adressen waren:

```
1 BME280_1 -> 0x76
2 BME280_2 -> 0x77
```

Im Serial Monitor wurde angezeigt:

```
1 I2C Geraete: [118, 119]
```

Das bedeutet:

```
1 118 = 0x76
2 119 = 0x77
```

Damit wurden beide BME280-Sensoren korrekt erkannt.

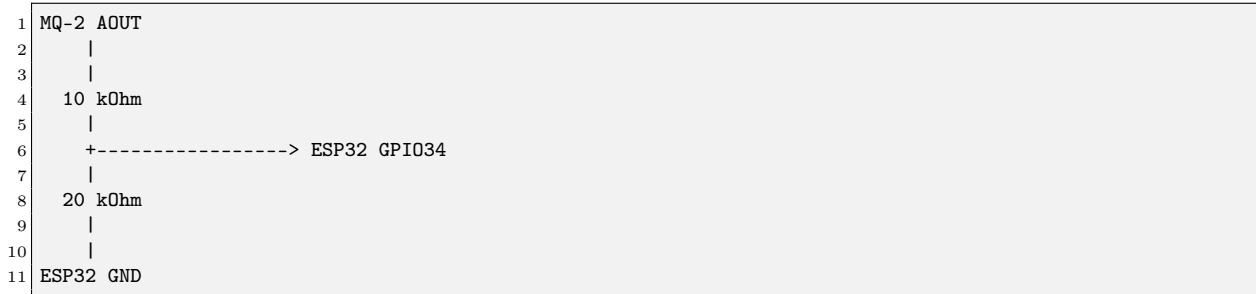
2.4.2 MQ-2

Der MQ-2 ist ein Gas- und Rauchsensor. Er eignet sich zur Erkennung von Rauch und brennbaren Gasen [6]. Im Projekt wird er über GPIO34 analog gelesen.

Wichtig ist, dass der ESP32 maximal 3.3V am ADC-Pin verträgt. Deshalb darf der MQ-2-Ausgang nicht direkt mit 5V auf GPIO34 verbunden werden. Der analoge Ausgang AOUT des MQ-2 muss deswegen über einen Spannungsteiler mit dem ESP32 verbunden werden. Der Spannungsteiler besteht aus einem Widerstand mit 10 kΩ und einem Widerstand mit

20 k Ω . Dabei wird der 10 k Ω -Widerstand zwischen dem MQ-2-Ausgang und dem Messpunkt angeschlossen. Der 20 k Ω -Widerstand wird zwischen dem Messpunkt und Masse verbunden. Der Messpunkt wird anschließend mit GPIO34 des ESP32 verbunden.

Die Verkabelung des Spannungsteilers lautet:



Die Ausgangsspannung des Spannungsteilers wird mit der Spannungsteilerformel berechnet:

$$U_{\text{GPIO34}} = U_{\text{AOUT}} \cdot \frac{R_2}{R_1 + R_2}$$

3 Implementierung

3.1 Blockschaltbild

Das folgende Blockschaltbild zeigt die wichtigsten Komponenten und deren Verbindung. Der ESP32 sendet echte Sensordaten per MQTT an HiveMQ Cloud. Das Railway Backend empfängt diese Daten, verarbeitet sie und stellt sie über eine REST API für das Streamlit Dashboard bereit.

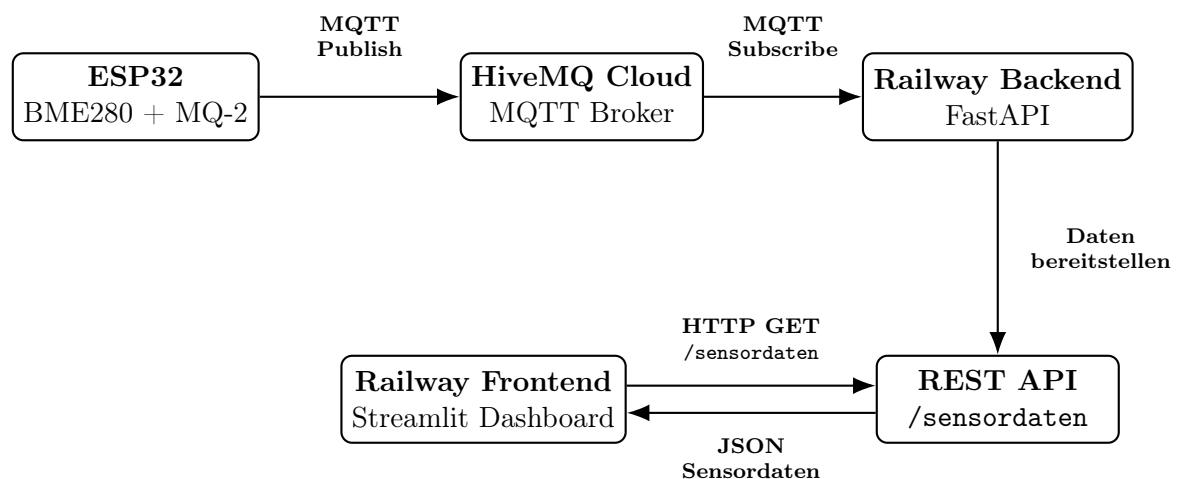


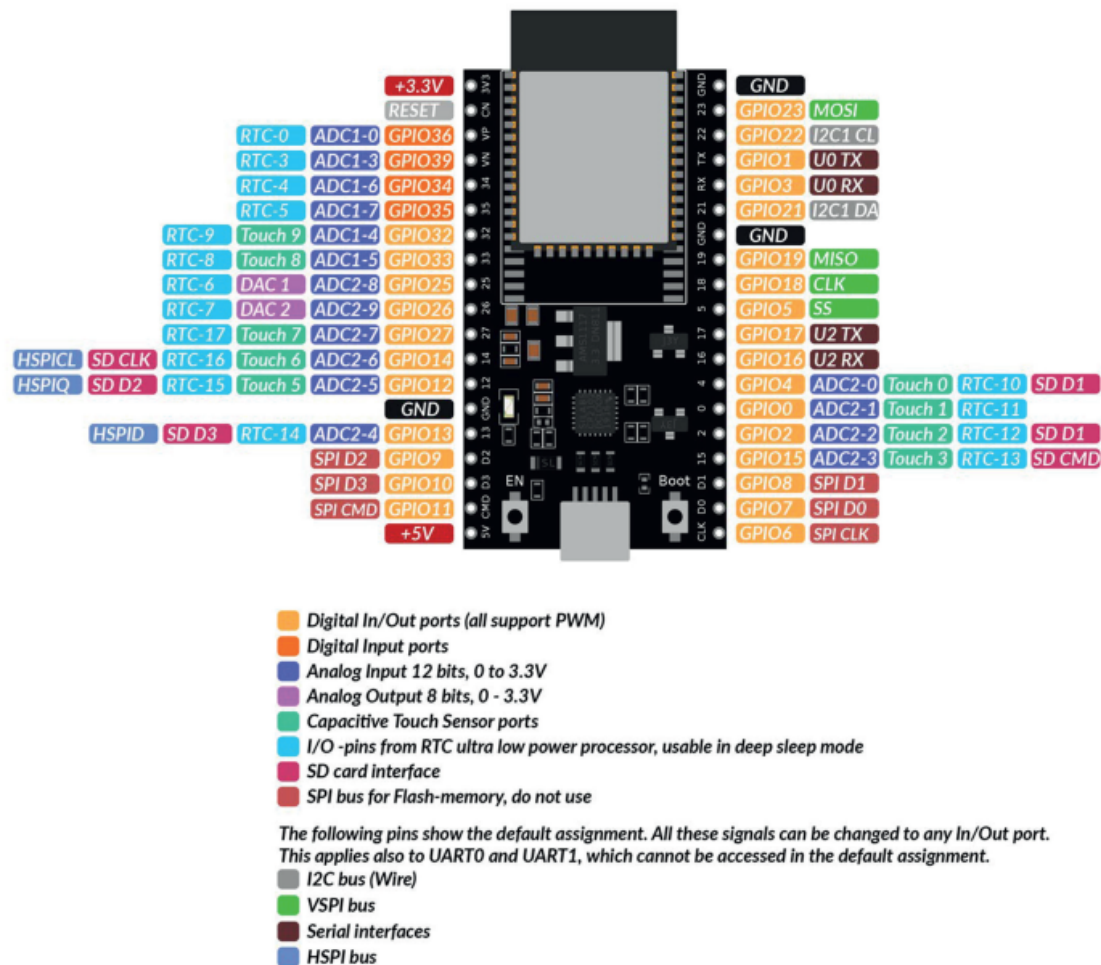
Abbildung 1: Blockschaltbild des IoT-Dashboard-Projekts

3.2 Hardware- und MicroPython-Setup

3.2.1 ESP32

Als zentrale Hardwarekomponente wurde ein ESP32 DevKit verwendet. Der ESP32 verfügt über integriertes WLAN sowie mehrere digitale und analoge Ein- und Ausgänge. Dadurch kann er die angeschlossenen BME280- und MQ-2-Sensoren auslesen und die Messdaten per MQTT an den Cloud-Broker übertragen.

Abbildung 2 zeigt die Pinbelegung des verwendeten ESP32-Boards.



3.2.2 BME280 Verkabelung

Die beiden BME280-Sensoren werden über den I2C-Bus mit dem ESP32 verbunden. Beide Sensoren verwenden dieselben SDA- und SCL-Leitungen, besitzen jedoch unterschiedliche I2C-Adressen.

Die Verkabelung lautet:

- | | | |
|---|----------------|-----------------|
| 1 | BME280 VCC/VIN | -> ESP32 3V3 |
| 2 | BME280 GND | -> ESP32 GND |
| 3 | BME280 SDA | -> ESP32 GPIO21 |
| 4 | BME280 SCL | -> ESP32 GPIO22 |

Abbildung 3 zeigt das verwendete BME280-Sensormodul mit Anschlusskabel.

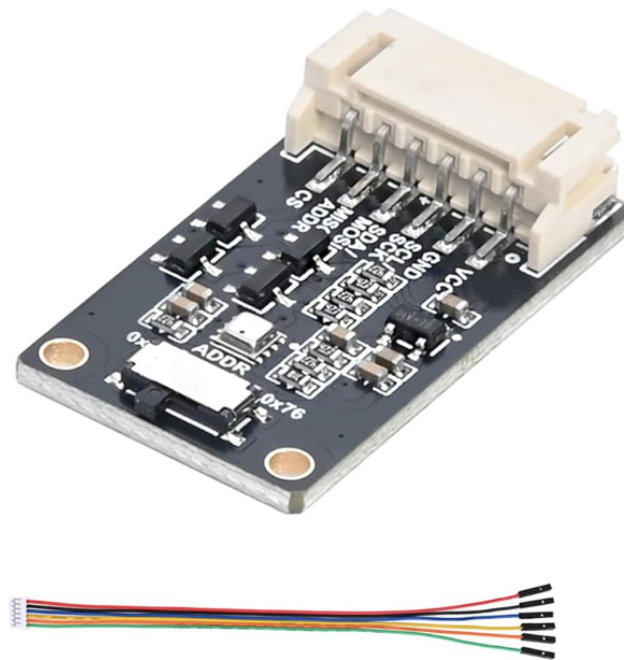


Abbildung 3: Verwendetes BME280-Sensormodul mit Anschlusskabel

Die beiden Sensoren besitzen folgende I2C-Adressen:

- | | | |
|---|----------|---------|
| 1 | BME280_1 | -> 0x76 |
| 2 | BME280_2 | -> 0x77 |

Dadurch können beide BME280-Sensoren gleichzeitig über GPIO21 und GPIO22 vom ESP32 ausgelesen werden.

3.2.3 MQ-2 Verkabelung

Der MQ-2-Sensor wird zur Gas- und Rauchüberwachung verwendet. Das Sensormodul wird mit 5V versorgt. Sein analoger Ausgang wird über einen Spannungsteiler mit dem ADC-Eingang GPIO34 des ESP32 verbunden.

Die grundlegende Verkabelung lautet:

- | | | |
|---|-----------|------------------------------------|
| 1 | MQ-2 VCC | -> ESP32 VIN/5V |
| 2 | MQ-2 GND | -> ESP32 GND |
| 3 | MQ-2 AOUT | -> Spannungsteiler -> ESP32 GPIO34 |

Abbildung 4 zeigt das verwendete MQ-2-Gas- und Rauchsensormodul.

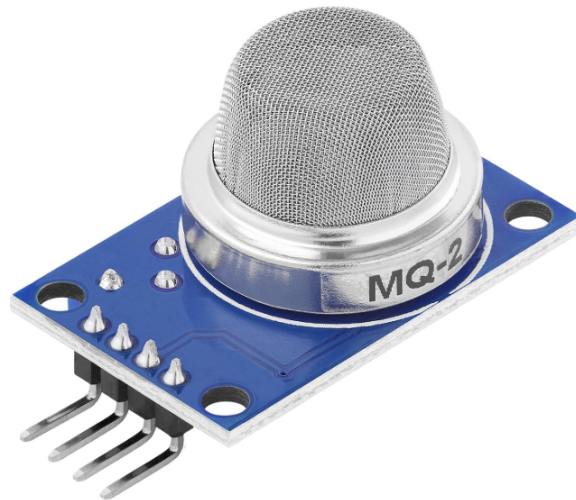


Abbildung 4: Verwendetes MQ-2-Gas- und Rauchsensormodul

3.2.4 MicroPython flashen

Damit der ESP32 das Sensorprogramm ausführen konnte, wurde zuerst MicroPython installiert. Dazu wurde der vorhandene Flash-Speicher gelöscht:

```
1 py -m esptool --chip esp32 --port COM3 --baud 115200 erase-flash
```

Danach wurde die MicroPython-Firmware auf den ESP32 übertragen:

```
1 py -m esptool --chip esp32 --port COM3 --baud 115200 write-flash --flash-mode dio --flash-freq 40m --flash-size 4MB -z 0x1000 .\ESP32_GENERIC-20260406-v1.28.0.bin
```

Nach dem erfolgreichen Flashen erschien der MicroPython-Prompt:

```
1 MicroPython v1.28.0
2 Generic ESP32 module with ESP32
3 >>>
```

Anschließend wurden die Projektdateien auf den ESP32 kopiert:

```
1 cd C:\Users\guedr\esp32-iot-hardware
2
3 py -m mpremote connect COM3 fs cp .\config.py :config.py
4 py -m mpremote connect COM3 fs cp .\bme280.py :bme280.py
5 py -m mpremote connect COM3 fs cp .\main.py :main.py
```

Die Datei `config.py` enthält die WLAN- und MQTT-Einstellungen. Die Datei `bme280.py` enthält den Treiber für die BME280-Sensoren. Die Datei `main.py` ist das Hauptprogramm des ESP32.

Zum Schluss wurde der ESP32 neu gestartet:

```
1 py -m mpremote connect COM3 reset
```

Nach dem Neustart wird `main.py` automatisch ausgeführt. Der ESP32 verbindet sich mit dem WLAN und dem MQTT-Broker, liest die Sensorwerte aus und sendet sie an das Backend.

Während der Übertragung musste der Serial Monitor geschlossen sein, da sonst die Schnittstelle COM3 blockiert war.

3.3 Docker Deployment

Für lokale Tests wurden die einzelnen Komponenten des IoT-Sensor-Dashboards als Docker-Container ausgeführt. Dadurch konnten MQTT-Broker, Backend, Frontend und Sensorsimulator unabhängig voneinander gestartet und verwaltet werden.

Folgende Docker-Container wurden verwendet:

```
1 mqtt
2 backend
3 frontend
4 sensor
```

Abbildung 5 zeigt die laufenden Container des IoT-Dashboards in Docker Desktop.

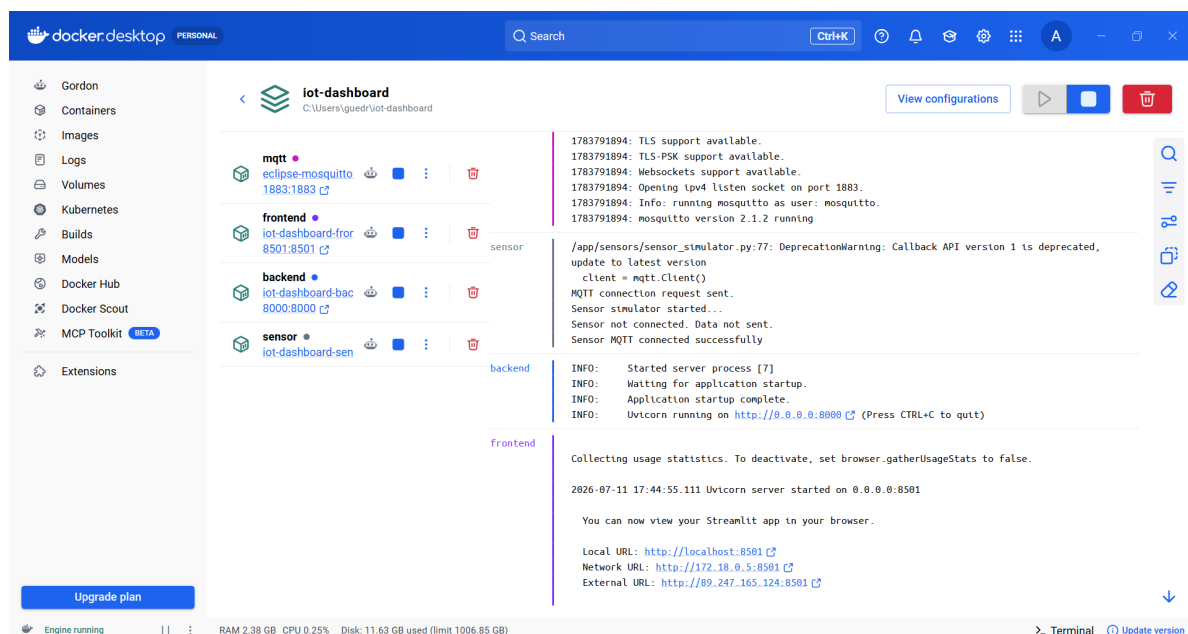


Abbildung 5: Lokales Docker-Deployment des IoT-Dashboards in Docker Desktop

Der lokale Start der benötigten Komponenten erfolgt mit folgenden Befehlen:

```
1 cd C:\Users\guedr\iot-dashboard
2 docker compose up -d mqtt backend frontend
3 docker compose stop sensor
4 docker compose ps
```

Der MQTT-Container verwendet Eclipse Mosquitto und stellt den lokalen MQTT-Broker über Port 1883 bereit. Das FastAPI-Backend ist über Port 8000 erreichbar. Das Streamlit-Frontend wird über Port 8501 bereitgestellt.

Der Sensorsimulator muss gestoppt werden, wenn echte Sensordaten vom ESP32 verwendet werden. Andernfalls würden gleichzeitig simulierte und echte Sensordaten an das System gesendet.

3.4 Cloud Deployment

Das Cloud Deployment wurde mit Railway und HiveMQ Cloud umgesetzt. Backend und Frontend wurden als getrennte Dienste auf Railway bereitgestellt. HiveMQ Cloud dient als zentraler MQTT-Broker für die Kommunikation zwischen dem ESP32 und dem Railway-Backend.

Folgende Cloud-Dienste wurden verwendet:

- **Backend:** Railway
- **Frontend:** Railway
- **MQTT-Broker:** HiveMQ Cloud

3.4.1 Railway Deployment

Auf Railway wurden zwei getrennte Dienste eingerichtet:

- Der Dienst **backend** führt die FastAPI-Anwendung aus.
- Der Dienst **IoT-Dashboard** führt das Streamlit-Frontend aus.

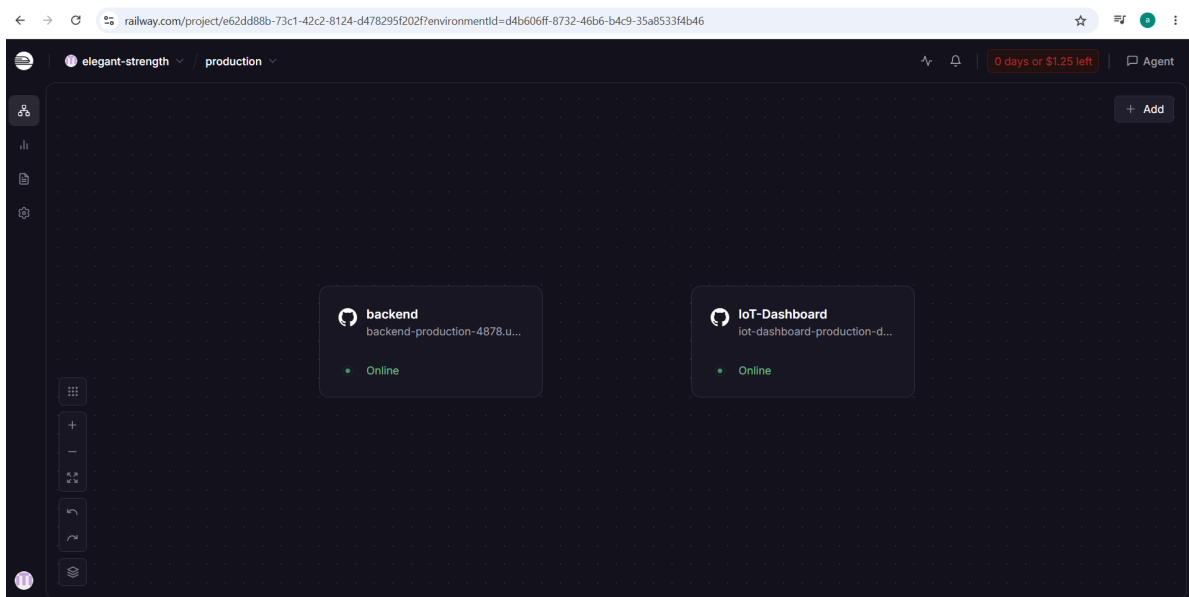


Abbildung 6: Cloud Deployment des Backends und Frontends auf Railway

Abbildung 6 zeigt das Railway-Projekt mit dem online ausgeführten Backend und Frontend.

Beide Railway-Dienste werden mit dem Status **Online** angezeigt. Das Backend empfängt die MQTT-Sensordaten und stellt sie über eine REST API bereit. Das Frontend ruft diese REST API auf und visualisiert die Sensordaten im Browser.

3.4.2 HiveMQ Cloud und MQTT Authentication

HiveMQ Cloud wird als öffentlicher MQTT-Broker verwendet. Der ESP32 sendet seine Sensordaten verschlüsselt über MQTT mit TLS an den Broker. Das Railway-Backend abonniert die Sensor-Topics und empfängt die veröffentlichten Messwerte.

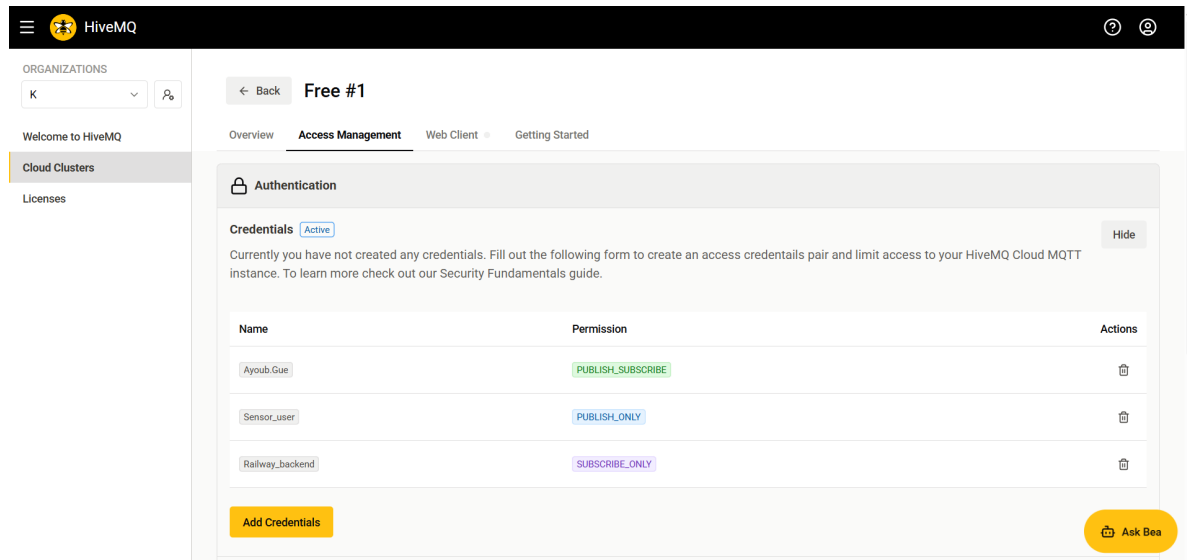


Abbildung 7: MQTT Authentication und Benutzerberechtigungen in HiveMQ Cloud

Abbildung 7 zeigt die in HiveMQ Cloud eingerichteten Benutzer und ihre Berechtigungen. Für die MQTT-Kommunikation wurden getrennte Benutzer mit unterschiedlichen Berechtigungen eingerichtet:

- **Sensor_user** besitzt die Berechtigung **PUBLISH_ONLY**. Dieser Benutzer wird vom ESP32 verwendet, um Sensordaten zu veröffentlichen.
- **Railway_backend** besitzt die Berechtigung **SUBSCRIBE_ONLY**. Dieser Benutzer wird vom Railway-Backend verwendet, um Sensordaten zu empfangen.
- Ein administrativer Benutzer besitzt die Berechtigungen zum Veröffentlichen und Abonnieren und wurde für Test- und Verwaltungsaufgaben verwendet.

Durch diese Aufteilung kann der ESP32 nur Daten senden, während das Backend nur Daten empfangen kann. Dadurch wird das Prinzip der minimal erforderlichen Berechtigungen umgesetzt und die MQTT-Kommunikation besser abgesichert.

Die finale Cloud-Datenkette lautet:

```

1 ESP32
2 -> MQTT Publish mit TLS
3 -> HiveMQ Cloud
4 -> MQTT Subscribe
5 -> Railway Backend
6 -> REST API
7 -> Railway Frontend

```

Das Dashboard ist unter folgender Adresse online erreichbar:

```

1 https://iot-dashboard-production-da48.up.railway.app

```

Das Backend ist unter folgenden Adressen erreichbar:

```
1 https://backend-production-4878.up.railway.app/health
```

Die Sensordaten können über folgenden REST-Endpunkt abgerufen werden:

```
1 https://backend-production-4878.up.railway.app/sensordaten
```

4 Ergebnisse

4.1 Echte Sensordaten

Der ESP32 konnte erfolgreich echte Sensordaten an HiveMQ Cloud senden. Das Backend empfing Daten von:

```
1 bme280_1
2 bme280_2
3 mq2_1
```

Damit funktionierte die Hauptkette:

```
1 ESP32 -> HiveMQ Cloud -> Railway Backend -> REST API -> Dashboard
```

Die angeschlossenen Sensoren lieferten erfolgreich reale Messwerte. Im Dashboard konnten die beiden BME280-Sensoren und der MQ-2-Sensor getrennt ausgewählt und angezeigt werden.

4.1.1 BME280-Sensoren

Die BME280-Sensoren erfassten Temperatur, Luftfeuchtigkeit und Luftdruck.

IoT Sensor Dashboard

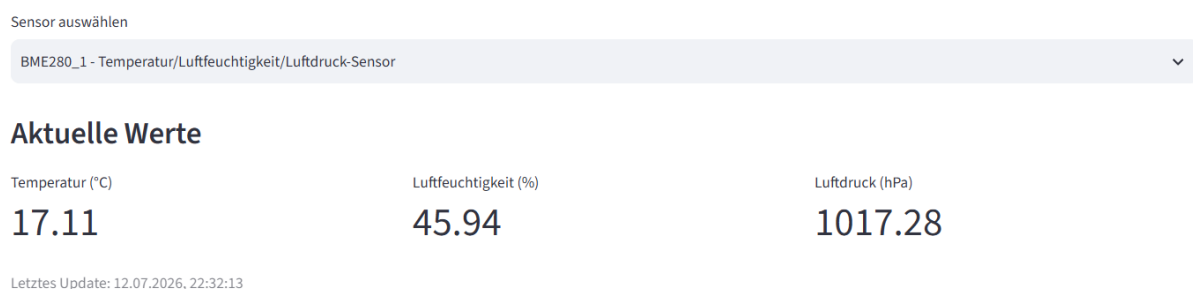


Abbildung 8: Aktuelle Messwerte des Sensors BME280_1

4.1.2 MQ-2-Sensor

Der MQ-2-Sensor erfasste den Gaswert und zeigte an, ob eine Rauchwarnung vorlag.

IoT Sensor Dashboard

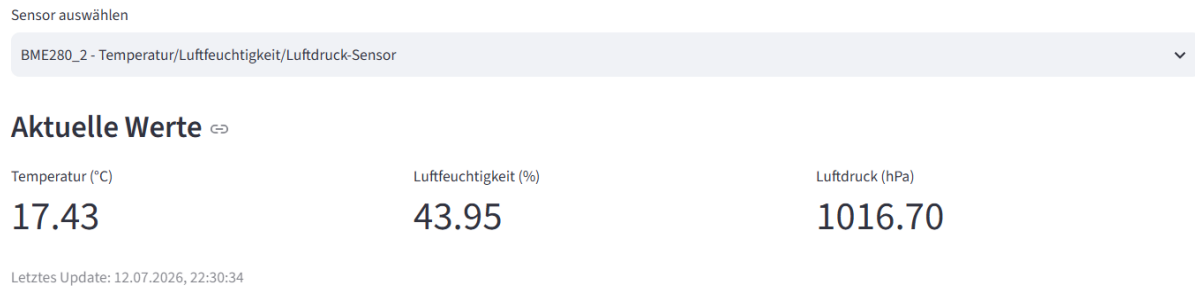


Abbildung 9: Aktuelle Messwerte des Sensors BME280_2

IoT Sensor Dashboard

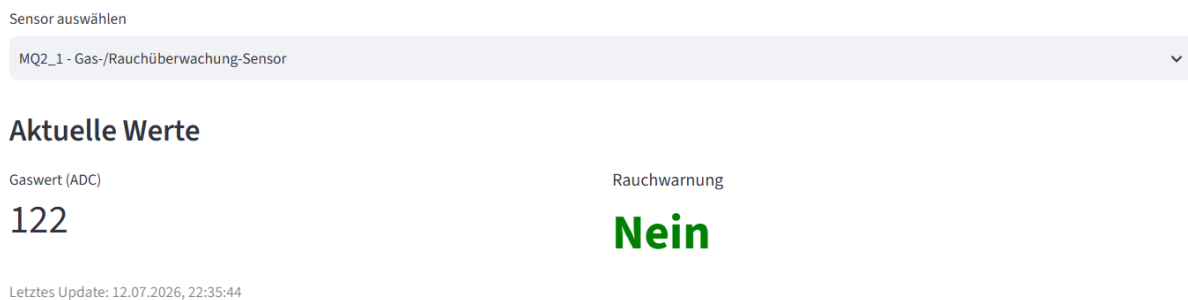


Abbildung 10: Aktueller Gaswert und Rauchwarnung des MQ-2-Sensors

4.1.3 Serial Monitor

Der Serial Monitor bestätigte, dass der ESP32 die Sensordaten regelmäßig an die MQTT-Topics sendete, wie die Abbildung 11 zeigt.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS MEMORY XTOOLS SERIAL MONITOR SETTINGS ABOUT
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - USB-SERIAL CH340 (COM3) Baud rate 115200
Line ending None [Stop Monitoring] [Fit] [Full Screen] [Close]
[{"sensor_type": "BME280", "gas_value": null, "temperature_c": 17.39, "sensor_id": "bme280_2", "humidity_perce
nt": 43.95, "pressure_hpa": 1016.67, "smoke_warning": false}
Gesendet an sensors/bme280_2
[{"sensor_type": "MQ2", "gas_value": 0, "temperature_c": null, "sensor_id": "mq2_1", "humidity_percent": null,
"pressure_hpa": null, "smoke_warning": false}
Gesendet an sensors/mq2_1
ESP32 (COM3)
Gesendet an sensors/bme280_1
[{"sensor_type": "BME280", "gas_value": null, "temperature_c": 17.05, "sensor_id": "bme280_1", "humidity_perce
nt": 40.12, "pressure_hpa": 1017.44, "smoke_warning": false}
Gesendet an sensors/bme280_2
[{"sensor_type": "BME280", "gas_value": null, "temperature_c": 17.43, "sensor_id": "bme280_2", "humidity_perce
nt": 44.42, "pressure_hpa": 1016.67, "smoke_warning": false}
Gesendet an sensors/mq2_1
[{"sensor_type": "MQ2", "gas_value": 0, "temperature_c": null, "sensor_id": "mq2_1", "humidity_percent": null,
"pressure_hpa": null, "smoke_warning": false}
ESP32 (COM3)
Gesendet an sensors/bme280_1
[{"sensor_type": "BME280", "gas_value": null, "temperature_c": 17.05, "sensor_id": "bme280_1", "humidity_perce
nt": 40.12, "pressure_hpa": 1017.42, "smoke_warning": false}
Gesendet an sensors/bme280_2
[{"sensor_type": "BME280", "gas_value": null, "temperature_c": 17.42, "sensor_id": "bme280_2", "humidity_perce
nt": 43.95, "pressure_hpa": 1016.72, "smoke_warning": false}
Gesendet an sensors/mq2_1
[{"sensor_type": "MQ2", "gas_value": 0, "temperature_c": null, "sensor_id": "mq2_1", "humidity_percent": null,
"pressure_hpa": null, "smoke_warning": false}
ESP32 (COM3)
Gesendet an sensors/bme280_1
[{"sensor_type": "BME280", "gas_value": null, "temperature_c": 17.05, "sensor_id": "bme280_1", "humidity_perce
nt": 41.72, "pressure_hpa": 1017.36, "smoke_warning": false}
Gesendet an sensors/bme280_2
[{"sensor_type": "BME280", "gas_value": null, "temperature_c": 17.43, "sensor_id": "bme280_2", "humidity_perce
nt": 45.7, "pressure_hpa": 1016.74, "smoke_warning": false}
Gesendet an sensors/mq2_1
[{"sensor_type": "MQ2", "gas_value": 0, "temperature_c": null, "sensor_id": "mq2_1", "humidity_percent": null,
"pressure_hpa": null, "smoke_warning": false}
ESP32 (COM3)
Type in a message to send to the serial port. Send as text Send Ctrl + C
```

Abbildung 11: Übertragung der Sensordaten im Serial Monitor

Damit funktionierte die Datenübertragung vom ESP32 bis zum Dashboard erfolgreich.

4.2 Mehrere Sensoren

Das Dashboard unterstützt mehrere Sensoren gleichzeitig. Über die Sensorauswahl kann zwischen den beiden BME280-Sensoren und dem MQ-2-Sensor gewechselt werden, wie die Abbildung 12 zeigt. Die Messwerte jedes Sensors werden getrennt angezeigt.

IoT Sensor Dashboard ⇄

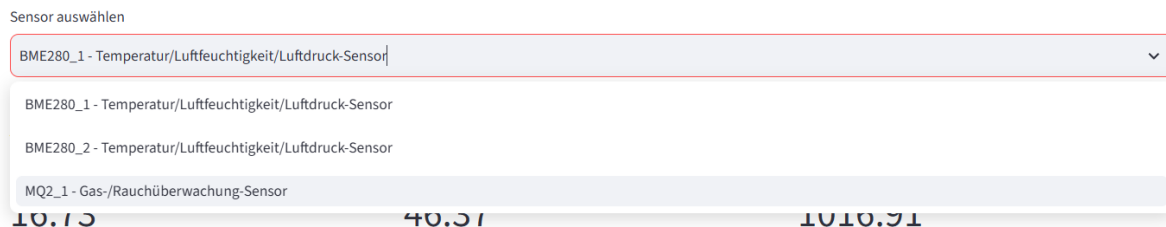


Abbildung 12: Auswahl der verfügbaren Sensoren im Dashboard

Jeder Sensor sendet seine Messwerte an ein eigenes MQTT-Topic:

```
1 sensors/bme280_1
2 sensors/bme280_2
3 sensors/mq2_1
```

Das Backend abonniert alle Sensor-Topics über:

```
1 sensors/#
```

Das Zeichen # ist ein MQTT-Wildcard-Zeichen. Es bedeutet, dass alle Untertopics unterhalb von `sensors/` empfangen werden. Dadurch konnte das Backend die Daten aller drei Sensoren gleichzeitig verarbeiten und im Dashboard getrennt darstellen.

4.3 Benutzer-Login

Das Dashboard ist durch einen Benutzer-Login geschützt. Erst nach der Eingabe gültiger Zugangsdaten wird der Zugriff auf die Sensordaten und Dashboard-Funktionen freigegeben, wie in Abbildung 13 dargestellt.

Der Benutzer-Login bietet folgende Vorteile:

- Schutz der Sensordaten vor unberechtigttem Zugriff
- kontrollierter Zugriff auf das Dashboard
- übersichtlichere Benutzeroberfläche
- Möglichkeit zum Abmelden über eine Logout-Funktion

Damit wurde zusätzlich zur MQTT-Sicherheit auch der Zugriff auf die grafische Benutzeroberfläche abgesichert.

Nach erfolgreicher Anmeldung zeigt das Dashboard den Status „Eingeloggt“ sowie eine Schaltfläche zum Abmelden, wie in Abbildung 14 dargestellt.

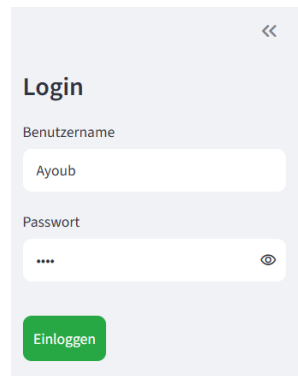


Abbildung 13: Login-Bereich des IoT-Dashboards



Abbildung 14: Erfolgreiche Anmeldung und Logout-Funktion

4.4 Echtzeit Auto Refresh

Das Dashboard aktualisiert die Sensordaten automatisch in festen Zeitintervallen. Dadurch müssen die Messwerte nicht manuell neu geladen werden.

Ein mögliches Aktualisierungsintervall ist:

1 Z.B. Auto Refresh: alle 5 Sekunden, wie in Abbildung 15 zeigt.

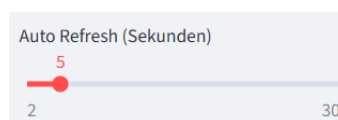


Abbildung 15: Auto-Refresh im Dashboard

Bei jeder Aktualisierung ruft das Streamlit-Frontend die aktuellen Daten über die REST API des Backends ab. Neue Messwerte werden anschließend automatisch in der Messtabelle, den Kennzahlen und den Diagrammen angezeigt.

Die automatische Aktualisierung bietet folgende Vorteile:

- neue Sensordaten erscheinen ohne manuelles Neuladen,
- das Dashboard funktioniert wie ein Live-Monitoring-System,
- Grenzwertwarnungen werden zeitnah angezeigt,
- Änderungen der Sensorwerte sind direkt sichtbar.

Die Auto-Refresh-Funktion wird erst nach erfolgreicher Anmeldung angezeigt.

4.5 MQTT Authentication

Die MQTT-Kommunikation wurde durch Benutzername und Passwort abgesichert. Für den ESP32 und das Railway-Backend wurden getrennte MQTT-Benutzer mit unterschiedlichen Berechtigungen eingerichtet.

```
1 Sensor_user    -> PUBLISH_ONLY
2 Railway_backend -> SUBSCRIBE_ONLY
```

Der Benutzer **Sensor_user** wird vom ESP32 verwendet. Er darf Sensordaten an den MQTT-Broker veröffentlichen, aber keine Topics abonnieren.

Der Benutzer **Railway_backend** wird vom Backend verwendet. Er darf die Sensor-Topics abonnieren und Messwerte empfangen, aber keine Sensordaten veröffentlichen.

Die Kommunikation mit HiveMQ Cloud erfolgt über TLS:

```
1 MQTT_PORT = 8883
2 TLS/SSL   = aktiviert
```

Durch die getrennten Benutzerrechte wurden folgende Sicherheitsziele erreicht:

- nur autorisierte Geräte können auf den MQTT-Broker zugreifen,
- der ESP32 darf ausschließlich Daten senden,
- das Backend darf ausschließlich Daten empfangen,
- unautorisierte oder gefälschte Sensordaten werden erschwert,
- Zugangsdaten werden über Konfigurations- oder Umgebungsvariablen verwaltet.

Nach der korrekten Konfiguration zeigte das Backend:

```
1 MQTT connected successfully
2 MQTT subscribed to sensors/#
```

Damit war die sichere Verbindung zwischen ESP32, HiveMQ Cloud und Railway-Backend erfolgreich hergestellt.

4.6 Historische Analyse

Das Dashboard berechnet Durchschnitts-, Minimal- und Maximalwerte der gespeicherten Sensordaten. Die Abbildung 16 zeigt Historische Analyse der BME280-Messwerte.

Historische Analyse

Ø Temperatur (°C)	Min Temperatur (°C)	Max Temperatur (°C)	Ø Luftfeuchtigkeit (%)	Min Luftfeuchtigkeit (%)	Max Luftfeuchtigkeit (%)
17.11	16.75	17.68	45.87	42.66	59.74

Abbildung 16: Historische Analyse der BME280-Messwerte

Für den MQ-2-Sensor werden Durchschnitt, Minimum und Maximum des Gaswertes dargestellt. Die Abbildung 17 zeigt Historische Analyse der MQ-2-Gaswerte.

Diese Werte sind nicht nur aktuelle Werte, sondern Statistiken aus gespeicherten Messwerten.

Historische Analyse

Ø Gaswert (ADC)

40.30

Min Gaswert (ADC)

0.00

Max Gaswert (ADC)

1310.00

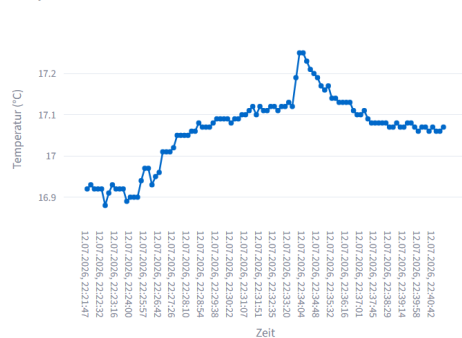
Abbildung 17: Historische Analyse der MQ-2-Gaswerte

4.7 Grafische Darstellung mit Diagrammen

Das Dashboard stellt die gespeicherten Sensordaten zusätzlich als Zeitverlauf dar. Dadurch können Veränderungen und auffällige Messwerte schnell erkannt werden.

Verlauf

Temperatur-Verlauf



Luftfeuchtigkeit-Verlauf

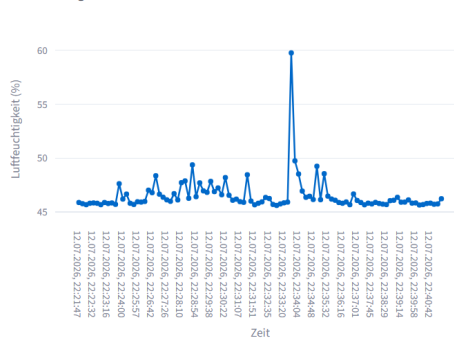
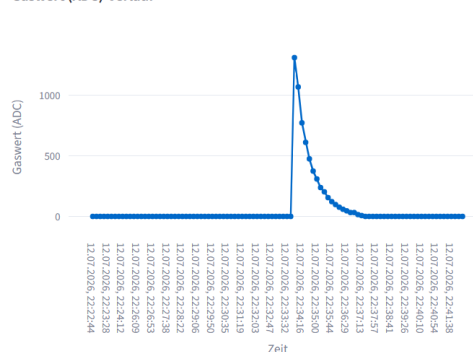


Abbildung 18: Temperatur- und Luftfeuchtigkeitsverlauf des BME280-Sensors

Die BME280-Diagramme zeigen den zeitlichen Verlauf von Temperatur und Luftfeuchtigkeit, wie in Abbildung 18 dargestellt.

Verlauf

Gaswert (ADC)-Verlauf



Rauchwarnung-Verlauf

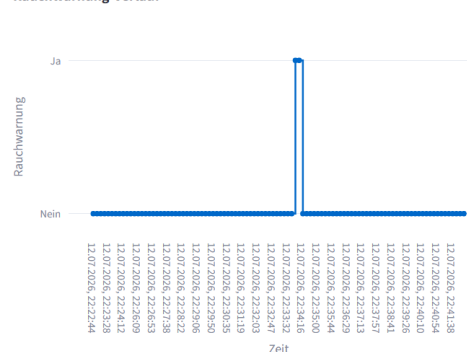


Abbildung 19: Gaswert- und Rauchwarnungsverlauf des MQ-2-Sensors

Die MQ-2-Diagramme zeigen Änderungen des Gaswertes sowie ausgelöste Rauchwarnungen, wie in Abbildung 19 dargestellt.

4.8 Messwert-Tabellen

Das Dashboard zeigt die Sensordaten zusätzlich zu den Diagrammen in tabellarischer Form. Die neuesten Messwerte stehen zuerst. Die UTC-Zeitstempel werden im Frontend automatisch in die deutsche Zeitzone **Europe/Berlin** umgewandelt.

4.8.1 BME280-Messwert-Tabelle

Die Tabelle des BME280-Sensors zeigt Sensorname, Zeitstempel, Temperatur, Luftfeuchtigkeit und Luftdruck.

Messwert-Tabelle

Nr.	Sensor	Zeit	Temperatur (°C)	Luftfeuchtigkeit (%)	Luftdruck (hPa)
0	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:46:36	17.09	47.04	1017.32
1	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:46:25	17.07	46.08	1017.31
2	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:46:14	17.09	45.96	1017.32
3	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:46:03	17.07	46.06	1017.28
4	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:45:52	17.06	46.15	1017.3
5	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:45:41	17.07	47.52	1017.29
6	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:45:30	17.08	46.24	1017.27
7	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:45:19	17.06	46.24	1017.32
8	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:45:08	17.05	47.02	1017.33
9	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-S	12.07.2026, 22:44:57	17.03	46.3	1017.35

Abbildung 20: Messwert-Tabelle des BME280-Sensors

4.8.2 MQ-2-Messwert-Tabelle

Die Tabelle des MQ-2-Sensors zeigt Sensorname, Zeitstempel, Gaswert und Rauchwarnung. In der dargestellten Messreihe lag keine Rauchwarnung vor.

Messwert-Tabelle

Nr.	Sensor	Zeit	Gaswert (ADC)	Rauchwarnung
0	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:45:31	0	Nein
1	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:45:20	0	Nein
2	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:45:09	0	Nein
3	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:44:58	0	Nein
4	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:44:46	0	Nein
5	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:44:35	0	Nein
6	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:44:24	0	Nein
7	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:44:13	0	Nein
8	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:44:02	0	Nein
9	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:43:51	0	Nein

Abbildung 21: Messwert-Tabelle des MQ-2-Sensors

Die Messwerte können zusätzlich als CSV- oder PDF-Datei exportiert werden.

4.9 Grenzwertwarnungen

Die Grenzwertwarnungen wurden im Projekt verwendet, um kritische Sensorwerte automatisch zu erkennen. Sobald ein Messwert einen festgelegten Grenzwert überschreitet, wird im Dashboard eine Warnung angezeigt. Zusätzlich kann das Backend eine E-Mail-Warnung auslösen.

Für den MQ-2-Gas- und Rauchsensor wurde die Rauchwarnung über einen Gas-Grenzwert berechnet:

```
1 GAS_WARNING_LIMIT = 3000
```

Logik:

```
1 gas_value <= 3000 -> smoke_warning = false
2 gas_value > 3000 -> smoke_warning = true
```

Zusätzlich wurden Grenzwertwarnungen für die Messwerte der BME280-Sensoren umgesetzt. Dabei werden Temperatur, Luftfeuchtigkeit und Luftdruck überwacht.

```
1 TEMPERATURE_WARNING_LIMIT = 30
2 HUMIDITY_WARNING_LIMIT = 70
3 PRESSURE_WARNING_LIMIT = 1030
```

Die Warnlogik lautet:

```
1 temperature_c > 30      -> Temperaturwarnung
2 humidity_percent > 70   -> Luftfeuchtigkeitwarnung
3 pressure_hpa > 1030     -> Luftdruckwarnung
4 gas_value > 3000       -> Gas-/Rauchwarnung
```

Damit erkennt das System folgende kritische Situationen:

- Temperatur über 30 °C
- Luftfeuchtigkeit über 70 %
- Luftdruck über 1030 hPa
- Gaswert über dem MQ-2-Grenzwert

Für Präsentation und Demo kann der MQ-2-Grenzwert reduziert werden:

```
1 GAS_WARNING_LIMIT = 1000
```

oder:

```
1 GAS_WARNING_LIMIT = 500
```

Dadurch kann die Warnfunktion einfacher getestet werden, ohne gefährliche Rauch- oder Gasquellen zu verwenden.

4.10 Machine-Learning-Vorhersage

Das Dashboard verwendet historische Messwerte, um zukünftige Sensorwerte für z.B. in der nächsten 60 Minuten vorherzusagen.

Machine-Learning-Vorhersage

Temperatur in 60 Min.

16.58 °C

Luftfeuchtigkeit in 60 Min.

49.80 %

Luftdruck in 60 Min.

1017.77 hPa

Abbildung 22: Machine-Learning-Vorhersage der BME280-Messwerte

Machine-Learning-Vorhersage

Gaswert in 60 Min.

149.98 ADC

Abbildung 23: Machine-Learning-Vorhersage des MQ-2-Gaswertes

Für den BME280-Sensor werden Temperatur, Luftfeuchtigkeit und Luftdruck vorhergesagt, wie die Abbildung 22 zeigt.

Für den MQ-2-Sensor wird der zukünftige Gaswert als ADC-Wert berechnet, wie die Abbildung 23 zeigt.

4.11 E-Mail-Alerts

Das Backend kann E-Mails versenden, wenn kritische Grenzwerte überschritten werden. Dafür wird die Resend API verwendet. Die E-Mail enthält Sensorname, Zeit, Messwerte und ausgelöste Warnungen.

Eine E-Mail-Warnung kann ausgelöst werden, wenn:

- der MQ-2-Gaswert den Grenzwert überschreitet,
- die Temperatur über 30 °C liegt,
- die Luftfeuchtigkeit über 70 % liegt,
- der Luftdruck über 1030 hPa liegt.

Zusätzlich werden E-Mail-Alerts auch für die Machine-Learning-Vorhersage verwendet. Wenn die Machine-Learning-Vorhersage zeigt, dass ein zukünftiger Sensorwert einen Grenzwert überschreiten wird, sendet das Backend ebenfalls eine Warn-E-Mail.

Dadurch erkennt das System nicht nur aktuelle kritische Messwerte, sondern kann auch auf vorhergesagte kritische Werte reagieren. Zum Beispiel kann eine E-Mail gesendet werden, wenn das Machine-Learning-Modell vorhersagt, dass die Temperatur in Zukunft über 30 °C steigen wird.

4.11.1 Temperaturwarnung

Bei einer Temperatur über 30 °C wurde eine Warnung angezeigt und eine E-Mail versendet. Die Abbildung 24 zeigt Temperaturwarnung und erfolgreicher E-Mail-Versand und die Abbildung 25 zeigt Empfangene E-Mail bei einer Temperaturüberschreitung.

IoT Sensor Dashboard

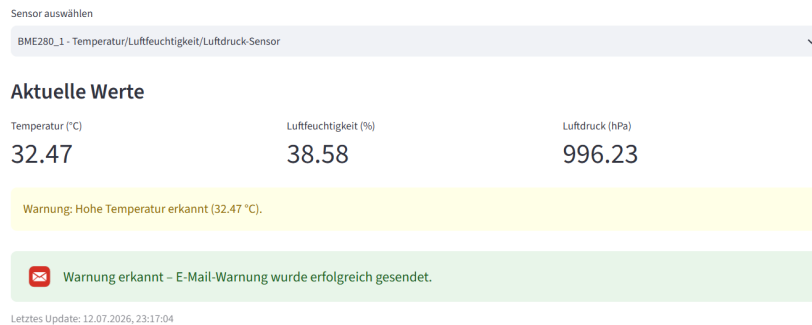


Abbildung 24: Temperaturwarnung und erfolgreicher E-Mail-Versand

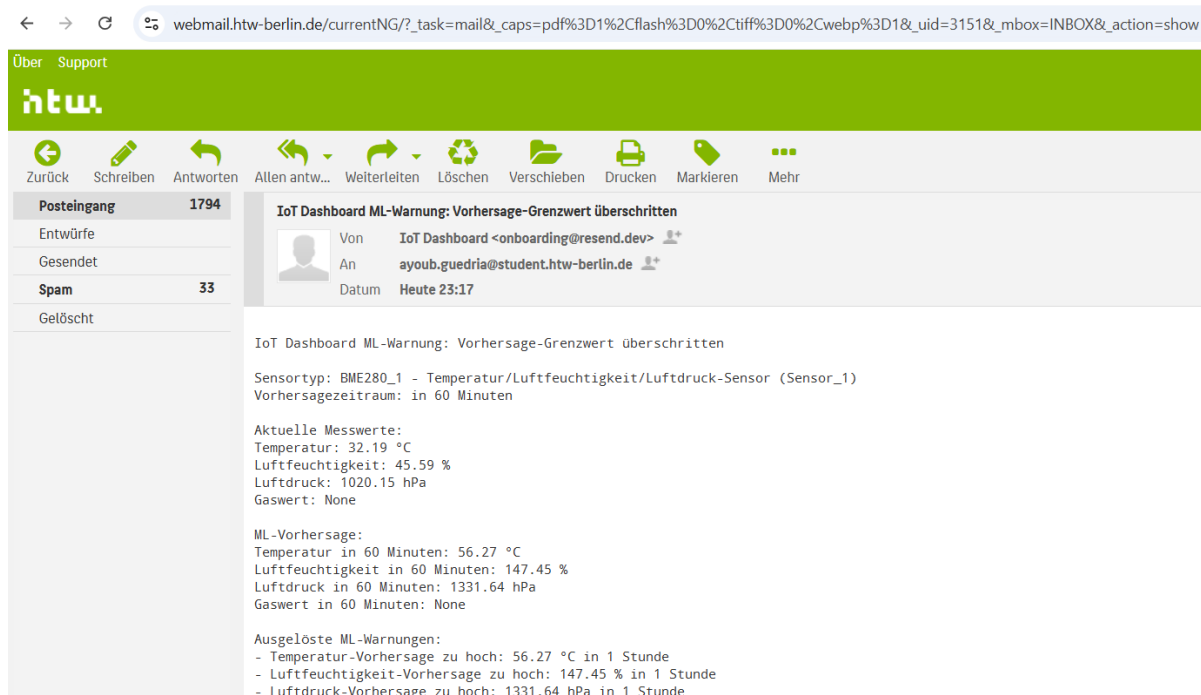


Abbildung 25: Empfangene E-Mail bei einer Temperaturüberschreitung

4.11.2 Gas- und Rauchwarnung

Bei einem erhöhten MQ-2-Gaswert wurde die Rauchwarnung aktiviert und ebenfalls eine E-Mail gesendet. Die Abbildung 26 zeigt Rauchwarnung und erfolgreicher E-Mail-Versand und die Abbildung 27 zeigt Empfangene E-Mail bei einer Rauchwarnung.

IoT Sensor Dashboard

Sensor auswählen

MQ2_1 - Gas-/Rauchüberwachung-Sensor

Aktuelle Werte

Gaswert (ADC)

1310

Rauchwarnung

Ja

Warnung: Rauchwarnung erkannt.



Warnung erkannt – E-Mail-Warnung wurde erfolgreich gesendet.

Letztes Update: 12.07.2026, 22:33:54

Abbildung 26: Rauchwarnung und erfolgreicher E-Mail-Versand

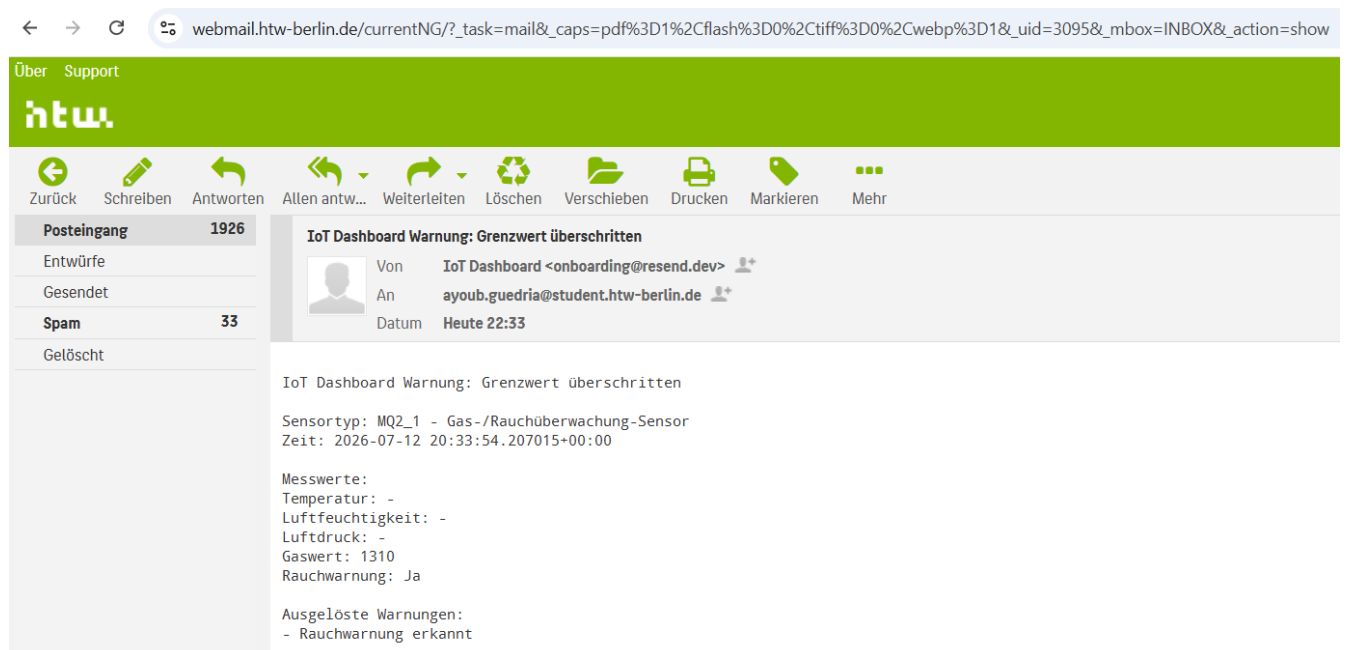


Abbildung 27: Empfangene E-Mail bei einer Rauchwarnung

4.11.3 Machine-Learning-Warnung

Auch bei einer vorhergesagten Grenzwertüberschreitung erzeugt das System eine Warnung und sendet eine E-Mail, wie die Abbildung 28 dargestellt.

Machine-Learning-Vorhersage

Gaswert in 60 Min.

3848.38 ADC

ML-Warnung erkannt: Gaswert-Vorhersage zu hoch: 3848.38 ADC in 1 Stunde

Abbildung 28: Machine-Learning-Warnung für einen zukünftigen Gaswert

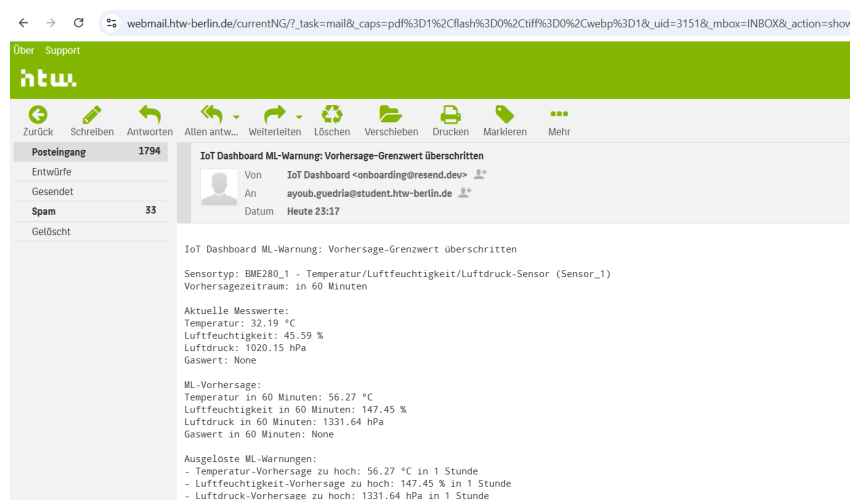


Abbildung 29: Empfangene E-Mail mit der Machine-Learning-Vorhersage

4.12 CSV/PDF-Export

Das Dashboard ermöglicht den Export der angezeigten Sensordaten als CSV- oder PDF-Datei. Dadurch können die Messwerte gespeichert, weiterverarbeitet und für die Dokumentation verwendet werden.

4.12.1 CSV-Export

Die CSV-Dateien enthalten die Zeitstempel und Messwerte in einer strukturierten, maschinenlesbaren Form, wie die Abbildungen 30 und 31 zeigen.

4.12.2 PDF-Export

Beim PDF-Export werden die Sensordaten übersichtlich in einer formatierten Tabelle dargestellt, wie die Abbildungen 32 und 33 zeigen.

```

bme280_1_aktuelle_messwert_tabelle (3).csv X
C: > Users > guedr > Downloads > bme280_1_aktuelle_messwert_tabelle (3).csv > data
1 Nr.,Sensor,Zeit,Temperatur (°C),Luftfeuchtigkeit (%),Luftdruck (hPa)
2 0,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:49:00",17.09,46.76,
3 1,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:48:49",17.12,46.38,
4 2,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:48:38",17.12,46.6,1
5 3,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:48:27",17.1,46.13,1
6 4,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:48:16",17.1,46.32,1
7 5,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:48:05",17.09,46.1,1
8 6,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:47:53",17.08,46.47,
9 7,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:47:42",17.08,46.42,
10 8,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:47:31",17.1,47.14,1
11 9,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:47:20",17.09,46.09,
12 10,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:47:09",17.1,46.07,
13 11,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:46:58",17.09,48.01
14 12,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:46:47",17.1,46.15,
15 13,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:46:36",17.09,47.04
16 14,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:46:25",17.07,46.08
17 15,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:46:14",17.09,45.96
18 16,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:46:03",17.07,46.06
19 17,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:45:52",17.06,46.15
20 18,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:45:41",17.07,47.52
21 19,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:45:30",17.08,46.24
22 20,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:45:19",17.06,46.24
23 21,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:45:08",17.05,47.02
24 22,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:44:57",17.03,46.3,
25 23,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:44:45",17.02,45.92
26 24,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:44:34",17.05,45.9,
27 25,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:44:23",17.05,46.13
28 26,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:44:12",17.05,46.17
29 27,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:44:01",17.06,47.04
30 28,BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor,"12.07.2026, 22:43:50",17.05,46.28

```

Abbildung 30: CSV-Export der BME280-Messwerte

```

bme280_1_aktuelle_messwert_tabelle (3).csv mq2_1_aktuelle_messwert_tabelle.csv X
C: > Users > guedr > Downloads > mq2_1_aktuelle_messwert_tabelle.csv > data
1 Nr.,Sensor,Zeit,Gaswert (ADC),Rauchwarnung
2 0,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:53:48",0,Nein
3 1,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:53:37",0,Nein
4 2,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:53:26",0,Nein
5 3,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:53:15",0,Nein
6 4,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:53:04",0,Nein
7 5,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:52:53",0,Nein
8 6,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:52:42",0,Nein
9 7,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:52:31",0,Nein
10 8,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:52:20",0,Nein
11 9,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:52:09",0,Nein
12 10,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:51:58",0,Nein
13 11,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:51:47",0,Nein
14 12,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:51:36",0,Nein
15 13,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:51:24",0,Nein
16 14,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:51:14",0,Nein
17 15,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:51:02",0,Nein
18 16,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:50:51",0,Nein
19 17,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:50:40",0,Nein
20 18,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:50:29",0,Nein
21 19,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:50:18",0,Nein
22 20,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:50:07",0,Nein
23 21,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:49:56",0,Nein
24 22,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:49:45",0,Nein
25 23,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:49:34",0,Nein
26 24,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:49:23",0,Nein
27 25,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:49:12",0,Nein
28 26,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:49:01",0,Nein
29 27,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:48:50",0,Nein
30 28,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:48:39",0,Nein
31 29,MQ2_1 - Gas-/Rauchüberwachung-Sensor,"12.07.2026, 22:48:28",0,Nein

```

Abbildung 31: CSV-Export der MQ-2-Messwerte

IoT Sensor Dashboard - Export für BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor

Nr.	Sensor	Zeit	Temperatur (°C)	Luftfeuchtigkeit (%)	Luftdruck (hPa)
0	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:47:20	17.09	46.09	1017.29
1	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:47:09	17.1	46.07	1017.36
2	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:46:58	17.09	48.01	1017.29
3	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:46:47	17.1	46.15	1017.36
4	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:46:36	17.09	47.04	1017.32
5	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:46:25	17.07	46.08	1017.31
6	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:46:14	17.09	45.96	1017.32
7	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:46:03	17.07	46.06	1017.28
8	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:45:52	17.06	46.15	1017.3
9	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:45:41	17.07	47.52	1017.29
10	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:45:30	17.08	46.24	1017.27
11	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:45:19	17.06	46.24	1017.32
12	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:45:08	17.05	47.02	1017.33
13	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:44:57	17.03	46.3	1017.35
14	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:44:45	17.02	45.92	1017.29
15	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:44:34	17.05	45.9	1017.33
16	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:44:23	17.05	46.13	1017.33
17	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:44:12	17.05	46.17	1017.25
18	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:44:01	17.06	47.04	1017.24
19	BME280_1 - Temperatur/Luftfeuchtigkeit/Luftdruck-Sensor	12.07.2026, 22:43:50	17.05	46.28	1017.22

Abbildung 32: PDF-Export der BME280-Messwerte

IoT Sensor Dashboard - Export für MQ2_1 - Gas-/Rauchüberwachung-Sensor

Nr.	Sensor	Zeit	Gaswert (ADC)	Rauchwarnung
0	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:52:42	0	Nein
1	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:52:31	0	Nein
2	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:52:20	0	Nein
3	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:52:09	0	Nein
4	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:51:58	0	Nein
5	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:51:47	0	Nein
6	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:51:36	0	Nein
7	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:51:24	0	Nein
8	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:51:14	0	Nein
9	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:51:02	0	Nein
10	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:50:51	0	Nein
11	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:50:40	0	Nein
12	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:50:29	0	Nein
13	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:50:18	0	Nein
14	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:50:07	0	Nein
15	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:49:56	0	Nein
16	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:49:45	0	Nein
17	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:49:34	0	Nein
18	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:49:23	0	Nein
19	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:49:12	0	Nein
20	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:49:01	0	Nein
21	MQ2_1 - Gas-/Rauchüberwachung-Sensor	12.07.2026, 22:48:50	0	Nein

Abbildung 33: PDF-Export der MQ-2-Messwerte

5 Herausforderungen

5.1 ESP32 Flash- und COM-Port-Probleme

Am Anfang hatte der ESP32 Boot- und Flash-Probleme. Außerdem war COM3 blockiert, wenn der Serial Monitor geöffnet war.

Lösung:

```
1 py -m esptool --chip esp32 --port COM3 --baud 115200 erase-flash
2 py -m esptool --chip esp32 --port COM3 --baud 115200 write-flash --flash-mode dio --flash-freq 40m --flash-size 4MB -z 0x1000 .\ESP32_GENERIC-20260406-v1.28.0.bin
```

Der Serial Monitor musste geschlossen werden, bevor `mpremote` verwendet wurde.

5.2 Lokaler MQTT-Broker nicht erreichbar

Der ESP32 sollte zuerst an den lokalen Mosquitto-Broker senden. Die Verbindung funktionierte aber nicht zuverlässig. Der Test zeigte:

```
1 TcpTestSucceeded : False
```

Lösung:

Es wurde HiveMQ Cloud statt lokaler MQTT-Verbindung verwendet.

5.3 MQTT Authentication

Das Backend zeigte zuerst:

```
1 MQTT connection failed. reason_code=Not authorized
```

Ursache:

Falscher Benutzer oder falsches Passwort im Railway Backend.

Lösung:

Das Backend musste `Railway_backend` verwenden, nicht `Sensor_user`.

5.4 Falsches MQTT Topic

Das Backend abonnierte zuerst:

```
1 sensors/data
```

Der ESP32 sendete aber an:

```
1 sensors/bme280_1
2 sensors/bme280_2
3 sensors/mq2_1
```

Lösung:

```
1 client.subscribe("sensors/#")
```

5.5 Fehlender Timestamp

Der ESP32 sendete keinen Timestamp. Das Backend-Modell verlangte aber einen Timestamp.

Lösung:

Das Backend ergänzt den Timestamp automatisch:

```
1 if "timestamp" not in payload or payload["timestamp"] is None:  
2     payload["timestamp"] = datetime.now(timezone.utc).isoformat()
```

5.6 Backend empfing Daten, aber /sensordaten war leer

Das Backend empfing MQTT-Nachrichten, speicherte sie aber zuerst nicht korrekt.

Lösung:

```
1 sensor_data_list.append(sensor_data)
```

oder bei SQLite:

```
1 INSERT INTO sensor_data (...)  
2 VALUES (...)
```

5.7 Dashboard leer trotz Backend-Daten

Das Backend hatte Daten, aber das Online-Dashboard blieb leer.

Ursache:

Falsche Backend-URL im Frontend.

Lösung:

```
1 BACKEND_URL=https://backend-production-4878.up.railway.app
```

5.8 Zeitdarstellung

Das Dashboard zeigte teilweise UTC-Zeit statt deutscher Zeit.

Lösung:

Die Zeit wurde im Frontend konsistent nach Deutschland/Berlin umgewandelt.

5.9 BME280 Temperaturabweichung

Der BME280 zeigte teilweise höhere Temperaturen als Handy oder Laptop.

Ursachen:

- Wärme vom ESP32
- Wärme vom MQ-2

- Nähe zu Laptop oder USB-Port
- Berührung mit der Hand
- schlechte Belüftung

Lösung:

- BME280 mindestens 30–50 cm vom ESP32 entfernen
- BME280 mindestens 30–50 cm vom MQ-2 entfernen
- 10–15 Minuten warten
- optional Temperatur-Offset setzen

Beispiel für Offset:

```
1 BME280 zeigt:      18 °C
2 Echte Temperatur:  15 °C
3 Offset:            15 - 18 = -3
```

5.10 MQ-2 Test und Sicherheit

Für MQ-2 wurde ein sicherer Test mit Alkohol-Dampf oder Desinfektionsmittel empfohlen.

```
1 1. MQ-2 5-10 Minuten aufwaermen lassen
2 2. Desinfektionsmittel oder Alkohol auf ein Tuch geben
3 3. Tuch 5-10 cm vor den MQ-2 halten
4 4. gas_value beobachten
5 5. Danach lueften
```

Nicht empfohlen wurden:

- offenes Feuer
- Feuerzeuggas
- Campinggas
- Butan
- Propan
- Benzin
- Aceton

5.11 E-Mail-Alerts

SMTP funktionierte online nicht zuverlässig. Deshalb wurde als Lösung eine API-basierte Variante wie Resend empfohlen.

5.12 CSV/PDF Export

Der Export zeigte zuerst nicht immer dieselben Daten wie die aktuelle Dashboard-Tabelle.

Lösung:

CSV und PDF müssen aus derselben gefilterten Tabelle erzeugt werden, die im Dashboard sichtbar ist.

6 Fazit und Ausblick

Das Projekt wurde erfolgreich als verteiltes IoT-System umgesetzt. Der ESP32 liest echte Sensordaten von zwei BME280-Sensoren und einem MQ-2-Sensor aus. Die Daten werden per MQTT an HiveMQ Cloud gesendet. Das Railway Backend empfängt und verarbeitet die MQTT-Nachrichten und stellt sie über eine REST API bereit. Das Railway Frontend visualisiert die Daten in einem Online-Dashboard.

Besonders wichtig war die schrittweise Fehleranalyse. Viele Probleme lagen nicht nur im Code, sondern in der Verbindung zwischen verteilten Komponenten. Beispiele dafür waren falsche MQTT-Zugangsdaten, falsche Topics, fehlende Timestamps, falsche Backend-URLs und falsche Zeitdarstellung.

Das Projekt demonstriert erfolgreich:

- echte Hardwareintegration
- MQTT-Kommunikation
- MQTT Authentication
- REST API
- Online Dashboard
- Auto Refresh
- historische Analyse
- Diagramme
- Grenzwertwarnungen
- E-Mail-Alerts
- CSV/PDF Export
- Docker Deployment
- Cloud Deployment
- Machine-Learning-Vorhersage

Für die Zukunft könnten weitere Erweiterungen umgesetzt werden:

- dauerhafte Datenbank statt In-Memory-Speicherung

- detailliertere ML-Modelle
- erweiterte Benutzerverwaltung
- weitere Sensoren
- bessere Kalibrierung des MQ-2
- Push-Benachrichtigungen zusätzlich zu E-Mail

Abbildungsverzeichnis

1	Blockschaltbild des IoT-Dashboard-Projekts	4
2	Pinbelegung des verwendeten ESP32 DevKit	5
3	Verwendetes BME280-Sensormodul mit Anschlusskabel	6
4	Verwendetes MQ-2-Gas- und Rauchsensormodul	7
5	Lokales Docker-Deployment des IoT-Dashboards in Docker Desktop	8
6	Cloud Deployment des Backends und Frontends auf Railway	9
7	MQTT Authentication und Benutzerberechtigungen in HiveMQ Cloud	10
8	Aktuelle Messwerte des Sensors BME280_1	11
9	Aktuelle Messwerte des Sensors BME280_2	12
10	Aktueller Gaswert und Rauchwarnung des MQ-2-Sensors	12
11	Übertragung der Sensordaten im Serial Monitor	12
12	Auswahl der verfügbaren Sensoren im Dashboard	13
13	Login-Bereich des IoT-Dashboards	14
14	Erfolgreiche Anmeldung und Logout-Funktion	14
15	Auto-Refresh im Dashboard	14
16	Historische Analyse der BME280-Messwerte	15
17	Historische Analyse der MQ-2-Gaswerte	16
18	Temperatur- und Luftfeuchtigkeitsverlauf des BME280-Sensors	16
19	Gaswert- und Rauchwarnungsverlauf des MQ-2-Sensors	16
20	Messwert-Tabelle des BME280-Sensors	17
21	Messwert-Tabelle des MQ-2-Sensors	17
22	Machine-Learning-Vorhersage der BME280-Messwerte	19
23	Machine-Learning-Vorhersage des MQ-2-Gaswertes	19
24	Temperaturwarnung und erfolgreicher E-Mail-Versand	20
25	Empfangene E-Mail bei einer Temperaturüberschreitung	20
26	Rauchwarnung und erfolgreicher E-Mail-Versand	21
27	Empfangene E-Mail bei einer Rauchwarnung	21
28	Machine-Learning-Warnung für einen zukünftigen Gaswert	22
29	Empfangene E-Mail mit der Machine-Learning-Vorhersage	22
30	CSV-Export der BME280-Messwerte	23
31	CSV-Export der MQ-2-Messwerte	23
32	PDF-Export der BME280-Messwerte	24
33	PDF-Export der MQ-2-Messwerte	24

Literatur

- [1] HiveMQ Cloud Dokumentation: *MQTT Cloud Broker und MQTT Authentication*. Online verfügbar: <https://www.hivemq.com/products/mqtt-cloud-broker/>
- [2] FastAPI Dokumentation: *REST API mit Python*. Online verfügbar: <https://fastapi.tiangolo.com/>
- [3] Streamlit Dokumentation: *Web-Dashboard mit Python*. Online verfügbar: <https://docs.streamlit.io/>
- [4] MicroPython Dokumentation: *MicroPython auf ESP32*. Online verfügbar: <https://docs.micropython.org/>
- [5] AZ-Delivery: *ESP32 Dev Kit C V4 Datenblatt / Anleitung*. Online verfügbar: https://cdn.shopify.com/s/files/1/1509/1638/files/AZ282_A_19-2_DE_B08BTS62L7.pdf?v=1719330230
- [6] AZ-Delivery: *MQ-2 Gas-/Rauchsensor Datenblatt / Anleitung*. Lokale Datei: file:///C:/Users/guedr/Downloads/AZ186_C_4-1_DE_B07CYYB82F.pdf
- [7] Bosch Sensortec: *BME280 Datasheet: Humidity, Pressure and Temperature Sensor*. Lokale Datei: file:///C:/Users/guedr/Downloads/BME280_Datasheet_Bosch.pdf
- [8] Guedria, Ayoub: *Python-Hauptprogramm des IoT Sensor Dashboard-Backends*. FastAPI, MQTT, SQLite, Resend E-Mail-Alerts und REST API, Projektdatei backend/main.py, 2026.

A ESP32 Befehle

A.1 MicroPython flashen

```
1 py -m esptool --chip esp32 --port COM3 --baud 115200 erase-flash
2
3 py -m esptool --chip esp32 --port COM3 --baud 115200 write-flash --flash-mode dio --flash-freq 40m --flash-size 4MB -z 0x1000 .\ESP32_GENERIC-20260406-v1.28.0.bin
```

A.2 Dateien auf ESP32 kopieren

```
1 cd C:\Users\guedr\esp32-iot-hardware
2
3 py -m mpremote connect COM3 fs cp .\config.py :config.py
4 py -m mpremote connect COM3 fs cp .\bme280.py :bme280.py
5 py -m mpremote connect COM3 fs cp .\main.py :main.py
6 py -m mpremote connect COM3 reset
```

B Docker und Backend Tests

B.1 Docker lokal starten

```
1 cd C:\Users\guedr\iot-dashboard
2 docker compose up -d mqtt backend frontend
3 docker compose stop sensor
4 docker compose ps
```

B.2 Backend testen

```
1 Invoke-RestMethod https://backend-production-4878.up.railway.app/sensordaten | Select-Object -Last 10
```

B.3 Streamlit testen

```
1 https://iot-dashboard-production-da48.up.railway.app/#ja
```

C Wichtige Konfigurationen

C.1 Railway Backend Variables

```
1 MQTT_BROKER=ec53cff1a655417a8246e130835c8427.s1.eu.hivemq.cloud
2 MQTT_PORT=8883
3 MQTT_USERNAME=Railway_backend
4 MQTT_PASSWORD=DEIN_RAILWAY_BACKEND_PASSWORT
```

C.2 Railway Frontend Variables

```
1 https://backend-production-4878.up.railway.app/health
2 https://backend-production-4878.up.railway.app/docs#/
```

D Python-Hauptprogramm: IoT Sensor Dashboard

Das Python-Hauptprogramm des Projekts befindet sich in der Datei `backend/main.py`. Diese Datei bildet die zentrale Backend-Komponente des IoT-Dashboards.

Sie implementiert:

- FastAPI-Anwendung
- MQTT-Verbindung zu HiveMQ Cloud
- MQTT Authentication mit Benutzername und Passwort
- Empfang von Sensordaten über `sensors/#`
- Validierung der Sensordaten mit Pydantic
- automatische Ergänzung fehlender Zeitstempel
- Speicherung der Sensordaten in SQLite
- REST-Endpunkte für aktuelle und historische Sensordaten
- E-Mail-Alerts über Resend API
- ML-Warn-E-Mail für Vorhersagewerte

D.1 Wichtige Ausschnitte aus dem Backend-Hauptprogramm

Listing 1: MQTT- und REST-Konfiguration im Backend

```
1 app = FastAPI(title="IoT Dashboard Backend", version="1.0.0")
2
3 MQTT_BROKER = os.getenv("MQTT_BROKER", "localhost")
4 MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
5 MQTT_TOPIC = os.getenv("MQTT_TOPIC", "sensors/#")
6 MQTT_USERNAME = os.getenv("MQTT_BACKEND_USERNAME") or os.getenv("MQTT_USERNAME")
7 MQTT_PASSWORD = os.getenv("MQTT_BACKEND_PASSWORD") or os.getenv("MQTT_PASSWORD")
```

Listing 2: Datenmodell für Sensordaten

```
1 class SensorData(BaseModel):
2     sensor_id: str
3     sensor_type: str
4     timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
5     temperature_c: Optional[float] = None
6     humidity_percent: Optional[float] = None
7     pressure_hpa: Optional[float] = None
8     gas_value: Optional[int] = None
9     smoke_warning: Optional[bool] = None
10    email_sent: bool = False
```

Listing 3: MQTT-Verbindung und Topic-Abonnement

```
1 def on_mqtt_connect(client, userdata, flags, reason_code, properties=None):
2     if reason_code.value != 0:
3         print(f"MQTT connection failed. reason_code={reason_code}")
4         return
5
6     print("MQTT connected successfully")
7     client.subscribe(MQTT_TOPIC, qos=0)
8     print("MQTT subscribed to sensors/#")
```

Listing 4: Automatische Timestamp-Ergänzung

```
1 payload = json.loads(msg.payload.decode())
2
3 if "timestamp" not in payload or payload["timestamp"] is None:
4     payload["timestamp"] = datetime.now(timezone.utc).isoformat()
5
6 sensor_data = SensorData(**payload)
```

Listing 5: REST-Endpunkte des Backends

```
1 @app.get("/health")
2 def health() -> dict:
3     return {"status": "ok"}
4
5 @app.get("/sensor-data", response_model=SensorData)
6 def get_latest_sensor_data() -> SensorData:
7     ...
8
9 @app.get("/history", response_model=list[SensorData])
10 @app.get("/sensordaten", response_model=list[SensorData])
11 def get_sensor_history() -> list[SensorData]:
12     ...
```